

BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Funções Built-in

Funções para Dicionários

```
# Dicionário para demonstração
vendas = {
    "janeiro": 15000,
    "fevereiro": 18000,
    "marco": 12000,
    "abril": 22000,
    "maio": 19000
}

# 1. len() - tamanho do dicionário
print(f"Quantidade de meses: {len(vendas)}") # 5

# 2. max() - valor máximo
maior_venda = max(vendas.values())
print(f"Maior venda: {maior_venda}") # 22000

# 3. min() - valor mínimo
menor_venda = min(vendas.values())
print(f"Menor venda: {menor_venda}") # 12000

# 4. sum() - soma dos valores
total_vendas = sum(vendas.values())
print(f"Total de vendas: {total_vendas}") # 86000

# 5. Média
media_vendas = total_vendas / len(vendas)
print(f"Média de vendas: {media_vendas:.2f}") # 17200.00
```



Funções de Busca

```
# Dicionário com dados complexos
alunos = {
    "Ana": {"notas": [8.5, 7.0, 9.2], "idade": 20},
    "Bruno": {"notas": [6.8, 8.0, 7.5], "idade": 19},
    "Carla": {"notas": [9.0, 8.5, 9.5], "idade": 21},
    "Diego": {"notas": [7.2, 6.5, 8.0], "idade": 20}
```

```
}

# 1. max() com função personalizada
melhor_aluno = max(alunos, key=lambda x: sum(alunos[x]["notas"])/len(alunos[x]["notas"]))
print(f"Melhor aluno: {melhor_aluno}")

# 2. min() com função personalizada
pior_aluno = min(alunos, key=lambda x: sum(alunos[x]["notas"])/len(alunos[x]["notas"]))
print(f"Pior aluno: {pior_aluno}")

# 3. sorted() - ordenação
alunos_ordenados = sorted(alunos, key=lambda x: sum(alunos[x]["notas"])/len(alunos[x]["notas"]))
print(f"Alunos ordenados por nota: {alunos_ordenados}")

# 4. filter() - filtragem
alunos_aprovados = list(filter(lambda x: sum(alunos[x]["notas"])/len(alunos[x]["notas"]) > 7.0, alunos))
print(f"Alunos aprovados: {alunos_aprovados}")
```



Funções de Agregação

```
# Dicionário com dados numéricos
produtos = {
    "notebook": {"preco": 2500, "estoque": 10},
    "mouse": {"preco": 50, "estoque": 25},
    "teclado": {"preco": 120, "estoque": 15},
    "monitor": {"preco": 800, "estoque": 8}
}

# 1. Soma de preços
total_precos = sum(produto["preco"] for produto in produtos.values())
print(f"Total de preços: R$ {total_precos}")

# 2. Soma de estoque
total_estoque = sum(produto["estoque"] for produto in produtos.values())
print(f"Total de estoque: {total_estoque} unidades")

# 3. Valor total do inventário
valor_inventario = sum(produto["preco"] * produto["estoque"] for produto in produtos.values())
print(f"Valor total do inventário: R$ {valor_inventario}")

# 4. Produto mais caro
produto_mais_caro = max(produtos, key=lambda x: produtos[x]["preco"])
print(f"Produto mais caro: {produto_mais_caro}")

# 5. Produto com maior estoque
produto_maior_estoque = max(produtos, key=lambda x: produtos[x]["estoque"])
print(f"Produto com maior estoque: {produto_maior_estoque}")
```



Funções de Transformação

```
# Dicionário original
```

```

dados = {
    "nome": "João Silva",
    "idade": 25,
    "salario": 5000,
    "departamento": "TI"
}

# 1. map() - transformação de valores
dados_upper = {k: v.upper() if isinstance(v, str) else v for k, v in dados.items()}
print("Dados em maiúsculo:", dados_upper)

# 2. filter() - filtragem de itens
dados_numericos = {k: v for k, v in dados.items() if isinstance(v, (int, float))}
print("Dados numéricos:", dados_numericos)

# 3. Transformação com função personalizada
def calcular_bonus(salario):
    return salario * 0.1

dados_com_bonus = {k: calcular_bonus(v) if k == "salario" else v for k, v in dados.items()}
print("Dados com bônus:", dados_com_bonus)

# 4. Inversão de chave-valor
dados_invertidos = {v: k for k, v in dados.items()}
print("Dados invertidos:", dados_invertidos)

```

Funções de Análise

```

# Dicionário com dados de vendas
vendas_mensais = {
    "janeiro": 15000,
    "fevereiro": 18000,
    "marco": 12000,
    "abril": 22000,
    "maio": 19000,
    "junho": 21000
}

# 1. Análise de tendência
valores = list(vendas_mensais.values())
crescimento = valores[-1] - valores[0]
print(f"Crescimento total: R$ {crescimento}")

# 2. Mês com maior crescimento
crescimento_mensal = {}
meses = list(vendas_mensais.keys())
for i in range(1, len(meses)):
    mes_atual = meses[i]
    mes_anterior = meses[i-1]
    crescimento_mensal[mes_atual] = vendas_mensais[mes_atual] - vendas_mensais[mes_anterior]

mes_maior_crescimento = max(crescimento_mensal, key=crescimento_mensal.get)
print(f"Mês com maior crescimento: {mes_maior_crescimento}")

# 3. Análise de variabilidade
media = sum(valores) / len(valores)
variancia = sum((x - media) ** 2 for x in valores) / len(valores)

```

```

desvio_padrao = variancia ** 0.5
print(f"Média: R$ {media:.2f}")
print(f"Desvio padrão: R$ {desvio_padrao:.2f}")

```

Exemplo: Sistema de Análise de Dados

```

# Sistema de análise de dados de funcionários
funcionarios = {
    "Ana": {"salario": 5000, "anos_experiencia": 3, "departamento": "TI"},
    "Bruno": {"salario": 4500, "anos_experiencia": 2, "departamento": "TI"},
    "Carla": {"salario": 6000, "anos_experiencia": 5, "departamento": "RH"},
    "Diego": {"salario": 4000, "anos_experiencia": 1, "departamento": "TI"},
    "Elena": {"salario": 5500, "anos_experiencia": 4, "departamento": "RH"}
}

# Análise por departamento
departamentos = {}
for nome, dados in funcionarios.items():
    dept = dados["departamento"]
    if dept not in departamentos:
        departamentos[dept] = {"funcionarios": [], "salarios": []}
    departamentos[dept]["funcionarios"].append(nome)
    departamentos[dept]["salarios"].append(dados["salario"])

# Estatísticas por departamento
for dept, info in departamentos.items():
    salarios = info["salarios"]
    print(f"\n{dept}:")
    print(f"  Funcionários: {len(info['funcionarios'])}")
    print(f"  Salário médio: R$ {sum(salarios)/len(salarios):.2f}")
    print(f"  Salário máximo: R$ {max(salarios)}")
    print(f"  Salário mínimo: R$ {min(salarios)}")

```

Dicas de Uso

- **max/min:** Use com `key=` para critérios personalizados
- **sum:** Funciona com valores numéricos
- **len:** Conta o número de pares chave-valor
- **sorted:** Retorna lista ordenada de chaves

